Guia Completo: Autenticação MSAL em Datadog Synthetic Tests via Injeção de Tokens (`loadExternalTokens`)

Este guia detalha a implementação da **Abordagem 2** para autenticar testes sintéticos do Datadog em aplicações que usam MSAL Browser, injetando tokens diretamente no cache do MSAL. Lembre-se que esta é uma **abordagem avançada**, destinada a cenários de teste específicos e com ressalvas importantes sobre sua complexidade e aplicabilidade.

Sumário:

1. **Pré-requisitos e Considerações Iniciais**
2. **Passo 1: Obtenção dos Tokens MSAL para Teste**
    - Métodos Possíveis (e suas limitações)
    - Informações Necessárias (Tokens, ClientInfo, AccountInfo)
3. **Passo 2: Configuração do Browser Test no Datadog**
    - Criação do Teste
    - Adição do Passo JavaScript
4. **Passo 3: Implementação do Script JavaScript para Injeção de Tokens**
    - Incluindo a Biblioteca MSAL (se necessário)
    - Instanciando `PublicClientApplication`
    - Estruturando os Dados para `loadExternalTokens`
    - Exemplo de Código Completo
5. **Passo 4: Execução e Validação do Teste**
6. **Boas Práticas e Cuidados de Segurança**
7. **Limitações e Alternativas**

(Conteúdo detalhado será adicionado nas próximas etapas)

1. Pré-requisitos e Considerações Iniciais

Antes de prosseguir, certifique-se de que:

- Você compreende as implicações de segurança e a complexidade desta abordagem.
- A Abordagem 1 (gravar o fluxo de login) foi considerada e considerada inviável para seu cenário específico.

- Você possui uma **conta de usuário dedicada** no Microsoft Entra ID (Azure AD) exclusivamente para estes testes sintéticos.
- Você tem acesso à configuração do `PublicClientApplication` da sua aplicação (Client ID, Authority, etc.).
- Você está ciente de que os tokens têm um tempo de vida limitado e precisará de um mecanismo para atualizá-los ou obtê-los periodicamente.

2. Passo 1: Obtenção dos Tokens MSAL para Teste

Este é frequentemente o passo mais desafiador. A API `loadExternalTokens` exige que você forneça os tokens e informações associadas antes de executar o teste no Datadog. Você precisa de um método externo ao Datadog para obter esses dados.

Informações Necessárias:

Para usar `loadExternalTokens` de forma eficaz, você geralmente precisará do seguinte (obtido para sua conta de teste dedicada):

1. **ID Token (`id_token`):** Contém informações sobre o usuário autenticado.
2. **Access Token (`access_token`):** Usado para acessar recursos protegidos (APIs). Você precisará do token e também:
    - `expires_in` : Tempo de vida do token em segundos.
    - `scope` : Escopos concedidos para este token.
    - `token_type` : Geralmente "Bearer".
    - `extended_expires_on` (Opcional): Tempo de expiração estendido.
3. **Refresh Token (`refresh_token`):** (Opcional, mas altamente recomendado para cenários de longa duração) Usado para obter novos ID e Access Tokens sem interação do usuário.
4. **Client Info (`client_info`):** Uma string codificada em Base64 que contém identificadores do tenant e do usuário, essencial para o MSAL identificar exclusivamente o usuário no cache.
5. **Account Information:** Necessário para associar os tokens ao usuário correto no cache.
    - `homeAccountId` : Identificador único da conta (formato: `uid.utid`).
    - `environment` : O domínio do provedor de identidade (ex: `login.microsoftonline.com`).
    - `tenantId` : O ID do tenant do Azure AD.
    - `username` : O nome de usuário (ex: `testuser@yourdomain.com`).
    - `localAccountId` : Identificador da conta no contexto local (geralmente o `oid` ou `sub` do token).

Métodos Possíveis para Obter os Tokens (e suas Limitações):

- **Método A: Extração Manual (Não recomendado para automação)**
    1. Faça login manualmente na sua aplicação com a conta de teste em um navegador.
    2. Abra as Ferramentas de Desenvolvedor (F12) e vá para a aba "Rede" (Network).
    3. Filtre pelas requisições para o endpoint de token do Azure AD (geralmente `login.microsoftonline.com/.../token` ou similar).
    4. Inspecione a resposta (response body) dessa requisição. Ela conterá `id_token`, `access_token`, `refresh_token`, `expires_in`, `scope`, `client_info`, etc.
    5. Decodifique o `id_token` (usando um site como jwt.ms ou jwt.io) para extrair informações da conta como `oid` (para `localAccountId`), `tid` (para `tenantId`), `preferred_username` (para `username`), e construa o `homeAccountId` (`oid.tid`).
    6. **Limitações:** Totalmente manual, impraticável para testes regulares, tokens expiram rapidamente, sujeito a erros.
- **Método B: Scripting com Fluxo ROPC (Resource Owner Password Credentials) - Fortemente Desencorajado**
    - Este fluxo permite obter tokens enviando diretamente usuário e senha para o endpoint de token. **A Microsoft desaconselha fortemente o uso do ROPC** devido a riscos de segurança (exposição de credenciais) e por não suportar identidades federadas ou MFA.
    - **Use apenas como último recurso em ambientes de teste altamente controlados, se permitido pela política de segurança e se MFA estiver desabilitado para a conta de teste.**
    - Você precisaria criar um script (Python, PowerShell, etc.) que faz uma requisição POST para `https://login.microsoftonline.com/{tenantId}/oauth2/v2.0/token` com `grant_type=password`, `client_id`, `scope`, `username`, e `password`.
    - **Limitações:** Inseguro, não funciona com MFA/federados, pode ser bloqueado pelo Azure AD, não reflete o fluxo real do usuário.
- **Método C: Serviço de Backend Auxiliar (Complexo)**
    - Criar um pequeno serviço de backend seguro que possa autenticar a conta de teste (talvez usando um fluxo mais seguro como Client Credentials para um service principal que representa o teste, ou outro método adequado) e expor

um endpoint seguro que o script do Datadog possa chamar para obter tokens frescos.
    - **Limitações:** Requer desenvolvimento e infraestrutura adicionais, complexidade na gestão de segurança deste serviço auxiliar.

- **Método D: Ferramentas de Teste E2E Existentes**
    - Se você já usa frameworks como Cypress, Playwright, etc., eles podem ter mecanismos ou plugins para lidar com a autenticação MSAL e potencialmente expor os tokens obtidos, que poderiam ser passados para o Datadog.
    - **Limitações:** Depende das capacidades da ferramenta e da sua configuração.

Recomendação: A obtenção segura e automatizada dos tokens é o principal obstáculo desta abordagem. Avalie cuidadosamente qual método (se algum) é viável e seguro no seu contexto. A extração manual serve apenas para testes pontuais e validação inicial.

---

3. Passo 2: Configuração do Browser Test no Datadog

Com os tokens e informações da conta em mãos (obtidos no Passo 1), você pode configurar o teste no Datadog para usar o script de injeção.

Passos:

1. **Navegue até Synthetic Tests:** No Datadog, vá para "UX Monitoring" > "Synthetic Tests".
2. **Crie um Novo Browser Test:** Clique em "New Test" > "Browser Test".
3. **Defina a URL Inicial:** Insira a URL da sua aplicação que você deseja testar após a autenticação. Como vamos injetar os tokens, não precisamos começar pela página de login.
4. **Configure Detalhes do Teste:** Dê um nome ao teste, selecione locais, frequência, etc., conforme necessário.
5. **Adicione o Passo JavaScript:** Este é o passo crucial.
    - Na seção "Test Steps", clique em "Add Step".
    - Selecione o tipo de passo: **"Advanced Actions" > "Execute JavaScript"**. Este passo deve ser o **primeiro** passo do seu teste, executado imediatamente após a navegação inicial para a URL definida.
    - Uma caixa de texto aparecerá para você inserir o código JavaScript. É aqui que você colocará o script detalhado no próximo passo (Passo 3) para carregar a biblioteca MSAL (se necessário) e chamar `loadExternalTokens`.

6. **Adicione Passos de Teste Pós-Injeção:** Após o passo "Execute JavaScript", adicione os passos normais do Browser Test para interagir com sua aplicação e realizar as validações desejadas (clicar em botões, verificar texto, etc.). A aplicação deve reconhecer o usuário como autenticado devido aos tokens injetados no cache do MSAL.
7. **Salvar o Teste:** Configure alertas e salve o teste.

Exemplo Visual (Conceitual da Interface Datadog):

```
+-----------------------------------------+
| Browser Test Configuration              |
+-----------------------------------------+
| Name: [Seu Teste com Injeção MSAL]      |
| Start URL: [URL da sua app pós-login]   |
| ... (Locations, Frequency, etc.)        |
+-----------------------------------------+
| Test Steps:                             |
|                                         |
| 1. Navigate to [URL da sua app pós-login]|
|                                         |
| 2. Execute JavaScript                   |
|    +-------------------------------+    |
|    | // Seu código JavaScript aqui |    |
|    | // (Detalhado no Passo 3)     |    |
|    | await injectMsalTokens(...);  |    |
|    +-------------------------------+    |
|                                         |
| 3. Click on element "#botao-principal"  |
| 4. Assert text "Bem-vindo, Usuário"     |
| ... (Outros passos de validação)        |
+-----------------------------------------+
```

Consideração: O script JavaScript será executado no contexto do navegador do Datadog Synthetic Test runner. Certifique-se de que qualquer dependência (como a própria biblioteca `@azure/msal-browser`) esteja acessível ou seja incluída/carregada dentro do seu script.

---

4. Passo 3: Implementação do Script JavaScript para Injeção de Tokens

Este script será inserido no passo "Execute JavaScript" do seu Browser Test no Datadog. Ele carregará a biblioteca MSAL Browser dinamicamente, configurará a instância e usará `loadExternalTokens` para injetar os dados obtidos no Passo 1.

Compatibilidade: Este exemplo é projetado para ser compatível com a API do `@azure/msal-browser` v2.31.0.

Script JavaScript:

```
 // Função auxiliar para carregar scripts dinamicamente
function loadScript(url) {
  return new Promise((resolve, reject) => {
    const script = document.createElement('script');
    script.src = url;
    script.async = true;
    // Resolve a promessa quando o script for carregado
    script.onload = () => {
        console.log(`Script loaded successfully: ${url}`);
        resolve();
    };
    // Rejeita a promessa em caso de erro
    script.onerror = () => {
        console.error(`Failed to load script: ${url}`);
        reject(new Error(`Failed to load script: ${url}`));
    };
    document.body.appendChild(script);
  });
}

// Função principal assíncrona para injetar os tokens
async function injectMsalTokens() {
  console.log("Iniciando injeção de tokens MSAL...");

  // --- Configuração e Dados de Exemplo ---
  // IMPORTANTE: Substitua estes placeholders pelos seus dados reais.
  // RECOMENDAÇÃO FORTE: NÃO codifique tokens sensíveis diretamente aqui.
  // Use variáveis globais do Datadog (ex: {{clientId}}) ou busque de um local seguro.

  const clientId = "YOUR_CLIENT_ID"; // Ex: Obtido de {{clientId}}
  const authority = "https://login.microsoftonline.com/YOUR_TENANT_ID"; // Ex:
Obtido de {{authority}}

  // Dados obtidos no Passo 1 (Obtenção dos Tokens)
  const idTokenToLoad = "PLACEHOLDER_ID_TOKEN";           // Ex: Obtido de
{{idToken}}
  const accessTokenToLoad = "PLACEHOLDER_ACCESS_TOKEN";      // Ex: Obtido de
{{accessToken}}
  const refreshTokenToLoad = "PLACEHOLDER_REFRESH_TOKEN";    // Opcional. Ex:
Obtido de {{refreshToken}}
  const clientInfoToLoad = "PLACEHOLDER_CLIENT_INFO";      // Ex: Obtido de
{{clientInfo}}
  const homeAccountIdToLoad = "PLACEHOLDER_HOME_ACCOUNT_ID"; // Ex:
Obtido de {{homeAccountId}}
  const environmentToLoad = "login.microsoftonline.com";   // Ou seu ambiente
específico
```

```
  const tenantIdToLoad = "YOUR_TENANT_ID";             // Deve corresponder à 'authority'
  const usernameToLoad = "testuser@yourdomain.com";      // Ex: Obtido de {{username}}
  const localAccountIdToLoad = "PLACEHOLDER_LOCAL_ACCOUNT_ID"; // Ex: Obtido de {{localAccountId}}
  // Substitua pelos escopos reais associados ao seu access_token
  const scopesToLoad = ["User.Read", "api://your-api/access"];
  const expiresInSeconds = 3599; // Tempo de expiração do access_token (em segundos)
  // const extendedExpiresOnSeconds = 6599; // Opcional: Se disponível

  // URL do CDN para MSAL Browser v2.31.0
  const msalCdnUrl = "https://alcdn.msauth.net/browser/2.31.0/js/msal-browser.min.js";

  try {
    // 1. Carregar a Biblioteca MSAL Browser Dinamicamente
    console.log(`Carregando MSAL Browser de ${msalCdnUrl}...`);
    await loadScript(msalCdnUrl);
    console.log("Biblioteca MSAL Browser carregada com sucesso.");

    // Verifica se o objeto 'msal' global foi carregado
    if (typeof msal === 'undefined' || !msal.PublicClientApplication) {
        throw new Error("Objeto MSAL não encontrado após carregar script. Verifique URL do CDN ou carregamento.");
    }

    // 2. Configurar e Instanciar PublicClientApplication
    // Use a mesma configuração da sua aplicação Angular
    const msalConfig = {
      auth: {
        clientId: clientId,
        authority: authority,
        // knownAuthorities: ["your_known_authority_domain"], // Se aplicável
        // redirectUri: "/", // Geralmente não necessário para injeção
      },
      cache: {
        // Configure a localização do cache se necessário (ex: sessionStorage)
        // cacheLocation: "sessionStorage",
        // storeAuthStateInCookie: false, // Se aplicável
      },
      // system: { loggerOptions: { ... } } // Opcional: para logs do MSAL
    };
    const pca = new msal.PublicClientApplication(msalConfig);
    console.log("PublicClientApplication instanciada.");

    // 3. Preparar os Dados para loadExternalTokens
    // Estrutura baseada na API do msal-browser v2.x (compatível com v2.31.0)

    const accountInfo = {
```

```
        homeAccountId: homeAccountIdToLoad,
        environment: environmentToLoad,
        tenantId: tenantIdToLoad,
        username: usernameToLoad,
        localAccountId: localAccountIdToLoad,
    };

    const silentRequest = {
        scopes: scopesToLoad,
        account: accountInfo,
        authority: authority, // Pode ser opcional se a conta for fornecida
    };

    const externalTokenResponse = {
        id_token: idTokenToLoad,
        access_token: accessTokenToLoad,
        refresh_token: refreshTokenToLoad, // Inclua se disponível
        token_type: "Bearer",
        scope: scopesToLoad.join(" "), // Escopos separados por espaço
        expires_in: expiresInSeconds,
        client_info: clientInfoToLoad, // Essencial para chave do cache
    };

    const loadTokenOptions = {
        // extendedExpiresOn: extendedExpiresOnSeconds // Opcional
    };

    console.log("Dados preparados para loadExternalTokens.");
    // Log para depuração (remova dados sensíveis em produção)
    // console.log("SilentRequest:", JSON.stringify(silentRequest, null, 2));
    // console.log("ExternalTokenResponse:", JSON.stringify(externalTokenResponse, null,
2));

    // 4. Chamar a API loadExternalTokens
    console.log("Chamando loadExternalTokens...");
    await pca.getTokenCache().loadExternalTokens(silentRequest,
externalTokenResponse, loadTokenOptions);

console.log("loadExternalTokens executado com sucesso. Tokens devem estar no
cache.");

    // 5. [Opcional] Verificar o Cache (para depuração)
    // Tenta obter um token silenciosamente para confirmar que o cache foi populado.
    try {
        console.log("Tentando acquireTokenSilent para verificar o cache...");
        // Use uma cópia da request para evitar mutações inesperadas
        const verificationRequest = { ...silentRequest };
        const silentResult = await pca.acquireTokenSilent(verificationRequest);
        // Log apenas uma parte do token por segurança
        console.log("acquireTokenSilent bem-sucedido após injeção. Access Token
(início):", silentResult.accessToken.substring(0, 10) + "...");
    } catch (error) {
```

```
      console.warn("Falha ao executar acquireTokenSilent após injeção. O cache pode não ter sido populado corretamente ou os dados/escopos estão incorretos.", error);
      // Isso pode ocorrer se escopos, conta ou tokens forem inválidos/expirados.
    }

  } catch (error) {
    console.error("Erro durante a injeção de tokens MSAL:", error);
    // Força o teste do Datadog a falhar se a injeção não funcionar
    throw new Error(`Falha na Injeção de Tokens MSAL: ${error.message || error}`);
  }
}

// Executa a função principal e retorna a Promise para que o Datadog aguarde
return injectMsalTokens();
```

Como Usar:

1. **Copie** este código.
2. **Cole** na caixa de texto do passo "Execute JavaScript" no seu Datadog Browser Test.
3. **Substitua** TODOS os valores `PLACEHOLDER_...` e `YOUR_...` pelos dados reais obtidos no Passo 1. **É crucial usar variáveis globais do Datadog ou um método seguro para buscar esses dados, em vez de codificá-los diretamente no script.**
4. **Ajuste** a configuração do `msalConfig` para corresponder exatamente à configuração usada na sua aplicação Angular (especialmente `clientId` e `authority`).
5. **Verifique** se os `scopesToLoad` correspondem aos escopos associados ao `accessTokenToLoad`.

Após este script ser executado com sucesso, o cache do MSAL no navegador do teste sintético estará populado, e os passos subsequentes do seu teste deverão encontrar o usuário como autenticado.

---

5. Passo 4: Execução e Validação do Teste

1. **Execute o Teste:** Salve o Browser Test no Datadog e execute-o manualmente ("Run Test Now") ou aguarde a execução agendada.
2. **Verifique os Logs:** Analise os logs do passo "Execute JavaScript". Procure pelas mensagens `console.log` que indicam o sucesso do carregamento do MSAL e da chamada `loadExternalTokens`. Verifique se há erros.
3. **Analise os Passos Subsequentes:** Verifique se os passos que interagem com a aplicação após a injeção de tokens funcionam como esperado (ex: acesso a páginas restritas, exibição de dados do usuário).

4. **Screenshots e Vídeo:** Use os screenshots e a gravação de vídeo do Datadog para depurar visualmente o que acontece durante a execução do script e nos passos seguintes.

6. Boas Práticas e Cuidados de Segurança

Ao usar a abordagem de injeção de tokens, é **fundamental** seguir estas práticas:

- **NÃO CODIFIQUE CREDENCIAIS OU TOKENS:** Jamais insira tokens, client secrets ou senhas diretamente no código JavaScript. Utilize as Variáveis Globais Obfuscadas do Datadog para armazenar e referenciar esses valores sensíveis (ex: `{{accessToken}}`, `{{clientInfo}}`).
- **CONTAS DE TESTE DEDICADAS:** Use sempre contas de usuário no Azure AD criadas **exclusivamente** para testes sintéticos. Essas contas devem ter o **mínimo privilégio necessário** para executar as ações do teste.
- **GERENCIAMENTO DE TOKENS:** Lembre-se que os tokens (especialmente Access Tokens) têm vida útil curta. Você precisará de um processo robusto e seguro (conforme discutido no Passo 1) para obter tokens atualizados regularmente e atualizar as variáveis globais do Datadog.
- **ESCOPO LIMITADO:** Solicite e use apenas os escopos (`scopes`) estritamente necessários para as ações do seu teste ao obter os tokens.
- **MONITORAMENTO E ALERTAS:** Configure alertas no Datadog para falhas no teste, o que pode indicar problemas com a injeção de tokens (ex: tokens expirados, script falhando).
- **LOGGING CUIDADOSO:** Evite logar tokens completos ou informações sensíveis nos `console.log` do seu script JavaScript, especialmente em ambientes de produção ou compartilhados. Use logs apenas para depuração e remova-os ou mascare dados sensíveis antes de salvar o teste final.
- **CONSISTÊNCIA DE VERSÃO:** Garanta que a versão da biblioteca `@azure/msal-browser` carregada via CDN no script corresponda à versão usada pela sua aplicação para evitar incompatibilidades sutis.
- **REVISÃO DE SEGURANÇA:** Discuta esta abordagem com sua equipe de segurança para garantir que ela esteja alinhada com as políticas da sua organização, dado que ela contorna o fluxo de login padrão.

7. Limitações e Alternativas

- **Complexidade:** Como visto, esta abordagem é mais complexa de configurar e manter do que gravar o fluxo de login.

- **Não Reflete o Fluxo Real:** O teste não valida o processo de login em si, apenas o comportamento da aplicação após a autenticação.
- **Dependência Externa:** Requer um mecanismo externo para obter e atualizar tokens.
- **Alternativa Principal:** A **Abordagem 1 (Gravar o Fluxo de Login)** continua sendo a mais recomendada sempre que possível, pois simula a experiência real do usuário de forma mais fiel e direta.

Fim do Guia